Roteiro de apresentação — 5 integrantes

Totem da Sorveteria · Architecture Challenge · Grupo 3 · UNIFOR · 03/11/2026

Tempo total: **15 minutos** + perguntas. Substitua `[Integrante N]` pelos nomes reais (aqui, na capa dos slides e em `docs/08-plano-de-trabalho.md`). As falas abaixo são as mesmas que estão nas **notas do apresentador** de cada slide (Keynote: *Visualizar → Mostrar notas do apresentador*).

Integrante	Assunto	Slides	Tempo
[Integrante 1]	Introdução e evolução do sistema	1, 2, 3	2 min
[Integrante 2]	Arquitetura e microsserviços	4, 5, 6, 7	3 min
[Integrante 3]	Design Patterns, SOLID e GRASP	8, 9, 10, 11	4 min
[Integrante 4]	Demonstração do sistema	12	4 min
[Integrante 5]	Testes, documentação e conclusão	13, 14, 15	2 min

Se o professor der menos tempo (ex.: 10 min): Integrante 1 fala só os slides 1–2 (1 min); Integrante 2 pula o slide 7 (2 min); Integrante 3 fala 1 padrão por slide (3 min); demonstração só com os passos 1, 3, 4 e 6 (3 min); Integrante 5 pula o slide 14 (1 min).

Integrante 1 — Introdução e evolução (slides 1–3 · 2 min)

Slide 1 — Capa (~20 s)

Bom dia/boa tarde. Somos o Grupo 3: [nomes]. Nosso projeto é o Totem da Sorveteria. Na atividade de Design Patterns, nós implementamos o totem como um programa de console em Java com cinco padrões. Neste desafio nós evoluímos esse mesmo sistema para uma arquitetura de quatro microsserviços em Spring Boot, com um front-end em React. O que vocês veem à direita é uma captura real da tela do totem funcionando.

Slide 2 — Mesmo domínio, arquitetura nova (~50 s)

O cenário continua o mesmo: o cliente escolhe a linha, monta o sorvete, usa o carrinho com cupom e desfazer, e paga. O enunciado pede evolução, não um sistema novo. Por isso mantivemos o domínio e as regras: preços, coberturas, o cupom SORVETE10 e o desfazer. A tabela mostra o que mudou: o console virou quatro microsserviços com API REST; os 5 padrões viraram 9; o dinheiro saiu de double para BigDecimal; agora cada serviço tem seu banco; criamos estoque e pagamento, que não existiam; e saímos de zero para 85 testes automatizados. Um detalhe: o sorvete zero de morango com calda e granulado custa R$ 17,50 tanto no projeto antigo quanto no novo — temos um teste que garante isso.

Slide 3 — Funcionalidades e diagramas UML (~50 s)

Este é o diagrama de casos de uso. O cliente escolhe a linha, monta o sorvete, adiciona e remove itens, aplica cupom, desfaz e finaliza o pedido. Finalizar inclui reservar estoque e processar o pagamento. O operador da loja consulta e repõe o estoque. Repare que cada caso de uso indica o padrão que o implementa — por exemplo, "Escolher linha" é a Abstract Factory. Também fizemos o diagrama de classes com os padrões, o diagrama de sequência do checkout e o diagrama de estados do pedido. Todos estão editáveis na pasta docs/diagramas. Agora o [Integrante 2] vai explicar a arquitetura.

Conceitos em uma frase

- *Evolução, não sistema novo*: o PDF do professor exige reaproveitar domínio, cenário e regras do trabalho anterior.
- *Caso de uso*: o que o usuário consegue fazer no sistema; «include» = sempre acontece junto; «extend» = acontece só em certas condições.

O que mostrar se pedirem

- Projeto antigo: `original/Grupo3_Sorveteria/sorveteria 2/src/sorveteria/Main.java` e `saida_exemplo.txt`.
- Diagramas: `docs/diagramas/casos-de-uso.drawio` (abre em diagrams.net) ou `ENTREGA_FINAL/diagramas_png/`.
- Teste do valor R$ 17,50: `catalogo-service/src/test/java/br/unifor/sorveteria/catalogo/domain/PadroesCriacionaisEEstruturaisTest.java` (método `reproduzOExemploDoProjetoOriginal`).

Perguntas prováveis

- *"O que vocês reaproveitaram do trabalho anterior?"* — O domínio (linhas Tradicional e Zero, coberturas, cupom, desfazer), os preços e os 5 padrões: Abstract Factory, Decorator, Command, Mediator e Memento. A cópia original está intacta na pasta `original/`.
- *"O que tinha de errado no projeto antigo?"* — Dinheiro em `double`, cupom fixo no código, eventos do Mediator como texto (String), nenhum teste e nenhuma persistência. Está em `docs/01-diagnostico-e-conformidade.md`.

Integrante 2 — Arquitetura e microsserviços (slides 4–7 · 3 min)

Slide 4 — Arquitetura geral (~45 s)

O enunciado exige no mínimo quatro microsserviços. Nós dividimos o sistema por propriedade de dados: cada serviço é dono de um conjunto de dados que muda por motivos diferentes. O front-end em React fala com os serviços pelo proxy do Vite, que funciona como um gateway de desenvolvimento. Três decisões sustentam isso. Primeiro, cada serviço tem o seu próprio banco H2: o pedido nunca lê a tabela do estoque, ele pergunta pela API. Segundo, a comunicação é REST com contratos explícitos e timeouts curtos, para o totem não travar. Terceiro, todos os serviços usam a mesma organização interna em camadas, a Service Layer — escolhemos isso em vez de MVC porque a "view" é o React, que está em outro processo.

Slide 5 — Os quatro microsserviços (~45 s)

São quatro serviços Spring Boot, cada um com a sua porta e o seu banco. O catálogo é dono do cardápio e dos preços, e monta o sorvete. O pedido cuida do carrinho, dos cupons, do desfazer e do checkout. O estoque guarda só SKU e quantidade — ele nem sabe preço. E o pagamento processa cartão e Pix de forma simulada e grava só os quatro últimos dígitos do cartão. Cada um pode ser compilado e executado sozinho. Se o estoque cair, o catálogo continua servindo o cardápio.

Slide 6 — Comunicação REST e o checkout distribuído (~50 s)

O ponto mais delicado da arquitetura é o checkout, porque ele envolve três serviços e não existe transação distribuída. O pedido-service orquestra uma saga: primeiro reserva o estoque, depois cobra, e por fim confirma. Se o pagamento é aprovado, a reserva é confirmada e o pedido fica PAGO. Se é recusado, a reserva é liberada — essa é a compensação — e o cliente pode tentar outro meio. E se a resposta do pagamento se perde, por exemplo num timeout, nós não sabemos se o dinheiro saiu; então o sistema não libera o estoque nem deixa cobrar de novo: o pedido fica em verificação até ser reconciliado. À direita está a resposta real de erro: todos os serviços usam o padrão RFC 7807, com um campo "codigo" que o front-end usa para decidir o que mostrar.

Slide 7 — Decisões arquiteturais (ADRs) (~40 s)

Registramos as decisões como ADRs: contexto, decisão e consequências — inclusive as negativas. As principais: dividir por dono dos dados; usar Service Layer; um banco H2 por serviço, que facilita a demonstração e troca para PostgreSQL só por configuração; manter o histórico de desfazer em memória, porque ele é da sessão do totem; usar uma saga com Mediator em vez de mensageria, porque o cliente está esperando a resposta na tela; e usar BigDecimal com idempotência no pagamento. Agora o [Integrante 3] apresenta os padrões de projeto.

Conceitos em uma frase

- *Microsserviço*: processo independente, com banco próprio, que expõe uma API.
- *Service Layer*: Controller (HTTP) → Service/Facade (caso de uso) → Domínio (regras) → Repository (banco).
- *Saga*: sequência de passos locais; se um falha, executam-se "compensações" (aqui: liberar a reserva).
- *Idempotência*: repetir a mesma chamada não muda o resultado (o mesmo pedido nunca é cobrado duas vezes).
- *RFC 7807*: formato padrão de erro em JSON (`status` , `detail` + nosso campo `codigo`).

O que mostrar se pedirem

- Portas e URLs: `pedido-service/src/main/resources/application.yml` (bloco `servicos:`).
- Contratos entre serviços: `pedido-service/src/main/java/br/unifor/sorveteria/pedido/client/dto/ContratosRemotos.java` .
- Saga: `pedido-service/.../checkout/TotemCheckoutMediator.java` (método `notificar`).
- Erros: `*/api/ApiExceptionHandler.java` em cada serviço.
- ADRs: `docs/05-adrs.md` .

Perguntas prováveis

- *"Isso não é um monólito dividido em portas?"* — Não: cada serviço tem banco próprio e nenhum lê a tabela de outro. O pedido só sabe o saldo perguntando à API do estoque (`EstoqueHttpClient.verificar`). O estoque nem conhece preços.
- *"Por que não usaram mensageria (Kafka/RabbitMQ)?"* — O cliente está parado no totem esperando a resposta; REST síncrono com compensação resolve com muito menos infraestrutura (ADR-006).
- *"E se um serviço cair?"* — Timeouts de 2 s/5 s; a resposta é 503 `SERVICO_INDISPONIVEL` com o nome do serviço; no checkout o estoque reservado é devolvido quando comprovadamente nada foi cobrado.
- *"Por que H2 e não PostgreSQL?"* — Zero instalação para a demonstração; o código JPA não muda, só a URL de conexão (ADR-004).

Integrante 3 — Design Patterns, SOLID e GRASP (slides 8–11 · 4 min)

Slide 8 — Padrões criacionais (~60 s)

Usamos nove padrões, três de cada categoria. Começando pelos criacionais. A Abstract Factory já existia: a FabricaZero cria a família inteira — sorvete e casquinha da linha Zero. Isso garante que nunca misturamos linhas. Os construtores dos produtos são package-private, então só a fábrica consegue criá-los. O Builder é novo: no projeto antigo a montagem estava espalhada dentro do mediador. Agora o SorveteBuilder recebe as partes e, no construir(), valida tudo: se o cliente escolher quatro coberturas, ele recusa — vocês vão ver isso na demonstração. O Factory Method fica no pagamento. Atenção à diferença: a Abstract Factory é um objeto que cria uma família; o Factory Method é um método, criarGateway(), que cada subclasse sobrescreve para criar um único produto.

Slide 9 — Padrões estruturais (~60 s)

Nos estruturais, o Decorator também veio do projeto original: cada cobertura embrulha o sorvete e soma o seu preço. Como o decorator implementa a mesma interface Sorvete, dá para empilhar: granulado em cima de calda em cima do sorvete. Agora ele também soma o item de estoque que a cobertura consome. O Adapter é novo: simulamos o SDK de uma operadora de cartão com uma interface que não controlamos — valores em centavos, respostas por código. O CartaoGatewayAdapter converte reais em centavos e traduz o código "51" para "Saldo insuficiente". Nenhuma outra classe conhece esse SDK. E a Facade: o controller do pedido tem uma linha por método. Toda a orquestração fica na PedidoFacade.

Slide 10 — Padrões comportamentais (~60 s)

Os três comportamentais vieram do projeto original e se completam. O Command transforma cada ação do carrinho — adicionar, remover, aplicar cupom — num objeto. Antes de agir, o comando pede um snapshot ao pedido: esse snapshot é o Memento. No desfazer, o comando manda o pedido restaurar a foto. O Memento é opaco: os getters são package-private, só o Pedido lê. Temos até um teste que verifica isso. O Mediator mudou de lugar: antes ele coordenava painéis de tela, que agora são do React. No back-end, o lugar onde várias peças reagem umas às outras é o checkout: quando o pagamento é recusado, o mediador manda o estoque liberar a reserva e o pedido registrar a recusa. Os participantes não se conhecem.

Slide 11 — SOLID e GRASP no código (~60 s)

O professor pediu que SOLID e GRASP fossem evidentes no código, então cada princípio aponta para uma classe real. Responsabilidade única: o SorveteBuilder só monta, o controller só traduz HTTP. Aberto/Fechado: para criar uma nova linha de sorvete, basta uma nova fábrica com @Component — a classe FabricasPorLinha recebe todas automaticamente, sem edição. Liskov: qualquer decorator substitui o sorvete. Segregação: um cliente pequeno por serviço. Inversão: as fábricas dependem de uma interface, não do banco. Do GRASP: o Pedido é o especialista nos próprios totais; a fábrica é a criadora dos produtos; os controllers só delegam; e o polimorfismo aparece nos tipos de cupom e nos processadores de pagamento. Agora o [Integrante 4] vai mostrar o sistema funcionando.

Conceitos em uma frase — ver a seção 1 do `Guia_Estudos_Perguntas.pdf` (os 9 padrões explicados com analogia simples).

Classes para mostrar (abrir no editor)

Padrão	Arquivo
Abstract Factory	`catalogo-service/.../catalogo/domain/abstractfactory/FabricaZero.java`
Builder	`catalogo-service/.../catalogo/domain/builder/SorveteBuilder.java` (método `validar`)
Factory Method	`pagamento-service/.../pagamento/domain/factorymethod/ProcessadorPagamento.java`
Decorator	`catalogo-service/.../catalogo/domain/decorator/CoberturaDecorator.java`
Adapter	`pagamento-service/.../pagamento/domain/adapter/CartaoGatewayAdapter.java`
Facade	`pedido-service/.../pedido/facade/PedidoFacade.java` × `pedido/api/PedidoController.java`
Command	`pedido-service/.../pedido/command/ComandoPedido.java`
Memento	`pedido-service/.../pedido/domain/SnapshotPedido.java` + `Pedido.java` (`criarSnapshot`, `restaurar`)
Mediator	`pedido-service/.../pedido/checkout/TotemCheckoutMediator.java`

Atalho para achar tudo: `grep -rn "PADRÃO:" */src/main` (cada arquivo de padrão começa com esse comentário).

Perguntas prováveis

- *"Qual a diferença entre Abstract Factory e Factory Method?"* — `FabricaLinha` é um **objeto** que cria uma **família** de dois produtos (sorvete + casquinha). O Factory Method é **um método** (`criarGateway()`) sobrescrito por **subclasses** de `ProcessadorPagamento`, criando **um** produto.
- *"O Command recebe o Pedido como parâmetro. Ainda é Command?"* — Sim. O comando continua encapsulando a ação e o estado para desfazê-la; a fachada continua sem saber como desfazer. Só mudou quando o receptor é ligado: em REST o pedido é recarregado do banco a cada requisição (comentário em `Comando.java`).

- *"A Facade não é só um Service comum?"* — Ela esconde um subsistema real: 3 clientes HTTP, comandos, histórico, repositórios, cupons e o mediador. O controller só conhece a fachada (`docs/03-design-patterns.md` §3.6).
- *"Repository do Spring conta como padrão?"* — Não contamos. Injeção de dependência, `JpaRepository` e proxies são do framework; os 9 padrões são classes nossas.
- *"Onde está o Open/Closed na prática?"* — Nova linha = nova classe `FabricaLinha` com `@Component`; `FabricasPorLinha` recebe a lista no construtor, sem `if`. Novo meio de pagamento = nova subclasse de `ProcessadorPagamento`.

Integrante 4 — Demonstração (slide 12 · 4 min)

Siga **`Roteiro_Demonstracao.md`** (cliques exatos, valores esperados e plano B). Fala do slide 12:

> 1. Escolho Zero Açúcar, morango, calda e granulado. A prévia mostra R$ 17,50 — quem calcula é o catálogo, com a Abstract Factory, o Builder e os Decorators.
> 2. Marco também chantilly e castanha: aparece "Máximo de 3 coberturas". Essa regra é do Builder, no back-end. Desmarco as duas.
> 3. Adiciono, escolho Limão e adiciono de novo: total R$ 31,00.
> 4. Aplico o cupom SORVETE10: R$ 27,90. O botão mostra "Desfazer: Aplicar cupom SORVETE10" — é o topo da pilha de comandos. Clico e o total volta para R$ 31,00: o Memento restaurou o estado.
> 5. Finalizo com o cartão recusado: o pagamento é recusado e a trilha mostra o Mediator liberando o estoque.
> 6. Finalizo com o cartão aprovado: pedido pago, com código de autorização.

Conceitos em uma frase

- *Prévia*: o front não calcula preço; cada escolha chama `POST /api/catalogo/montagens`.
- *Trilha do checkout*: lista de eventos que o Mediator devolve ao front (ex.: `ESTOQUE_RESERVADO → PAGAMENTO_RECUSADO → ESTOQUE_LIBERADO`).

Perguntas prováveis

- *"Esse preço é calculado no navegador?"* — Não. `NovoItemRequest` nem tem campo de preço; o pedido-service pede o preço oficial ao catálogo.
- *"E se eu pagar duas vezes?"* — O pedido pago fica bloqueado (409) e o pagamento-service devolve o pagamento já aprovado, sem cobrar de novo — inclusive com cliques simultâneos (`PagamentoConcorrenteTest`).

Integrante 5 — Testes, documentação e conclusão (slides 13–15 · 2 min)

Slide 13 — Testes automatizados e resultados (~50 s)

Usamos JUnit 5 com AssertJ e Mockito, e as ferramentas de teste do Spring. São 85 testes, todos passando: unitários dos padrões e das regras de dinheiro, testes de integração que sobem cada serviço com banco, testes de contrato da comunicação entre os serviços, e um teste de concorrência que garante que duas cobranças simultâneas do mesmo pedido geram um só pagamento. Além disso, um script de ponta a ponta faz 22 verificações com os quatro serviços reais no ar: compra aprovada, recusada, operadora fora do ar e falta de estoque. As saídas dessas execuções estão salvas na pasta de evidências.

Slide 14 — Limitações conhecidas e próximos passos (~35 s)

Também deixamos claro o que o sistema ainda não faz. Os bancos são em memória, o histórico de desfazer é da sessão, um pedido com pagamento incerto exige decisão manual se não houver aprovação, a proteção contra cobrança dupla vale para uma instância, e não há autenticação. Cada limitação está documentada num ADR ou no backlog, com o próximo passo — como expirar reservas pendentes e criar testes automatizados do front-end.

Slide 15 — Conclusão (~35 s)

Para concluir: nós evoluímos o totem do Grupo 3 para quatro microsserviços com banco próprio e um front-end que consome as APIs; os nove padrões aparecem no fluxo real de uma compra e são cobertos por testes; e as falhas — recusa, falta de estoque, serviço fora do ar e resultado incerto do pagamento — têm um tratamento definido. Obrigado. Estamos à disposição para perguntas.

Conceitos em uma frase

- *Teste unitário*: testa uma classe sozinha (ex.: `PedidoTest`).
- *Teste de integração*: sobe o serviço inteiro com banco e chama a API (`*ApiIntegrationTest`, MockMvc).
- *Teste de contrato*: simula o outro serviço e confere o JSON enviado/recebido (`ContratosHttpTest`, MockRestServiceServer).
- *Ponta a ponta*: os 4 serviços de verdade no ar (`scripts/smoke-test.sh`).

O que mostrar se pedirem

- `ENTREGA_FINAL/evidencias/mvn-test_resumo.txt` e `smoke-test_saida.txt` (execuções reais).
- Ao vivo (≈ 20 s): `mvn test` na raiz do projeto.
- Documentação: `README.md` e `docs/` (arquitetura, padrões, SOLID/GRASP, ADRs, API, testes, plano de trabalho).

Perguntas prováveis

- *"Como vocês testaram a comunicação entre os serviços sem subir tudo?"* — `ContratosHttpTest` usa `MockRestServiceServer` para conferir o JSON enviado e a interpretação de 201/409/422/503/erro de rede. O teste com tudo de pé é o `smoke-test.sh`.
- *"O front-end tem testes?"* — Não automatizados (limitação declarada, item B23). O fluxo foi percorrido num Chrome real por script e as capturas estão em `ENTREGA_FINAL/capturas/`.

- *"Quem fez o quê?"* — Tabela de responsabilidades em `docs/08-plano-de-trabalho.md` (preencher com os nomes reais).